

08 — Query Rewriting and Anti-Patterns

"The best optimization is often not an index — it's rewriting the query itself."

Table of Contents

1. [Learning Objectives](#)
 2. [Anti-Pattern Taxonomy](#)
 3. [N+1 Query Pattern](#)
 4. [SELECT * Anti-Pattern](#)
 5. [Functions on Indexed Columns](#)
 6. [OR vs UNION ALL for Index Usage](#)
 7. [NOT IN vs NOT EXISTS vs LEFT JOIN](#)
 8. [COUNT\(*\) vs COUNT\(column\) vs EXISTS](#)
 9. [Correlated Subqueries](#)
 10. [Excessive DISTINCT](#)
 11. [Using LIMIT without ORDER BY](#)
 12. [STRING AGG / ARRAY AGG Performance](#)
 13. [LIKE '%prefix%' and Substring Search](#)
 14. [Pagination Anti-Pattern \(OFFSET\)](#)
 15. [CTE Optimization Fence](#)
 16. [Common Mistakes](#)
 17. [Best Practices](#)
 18. [Interview Questions & Answers](#)
 19. [Exercises with Solutions](#)
 20. [Production Scenarios](#)
 21. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Recognize the most common SQL anti-patterns that cause performance problems
 - Rewrite N+1 queries into efficient JOINS
 - Fix function-on-column patterns that prevent index usage
 - Choose the correct approach for NOT IN / NOT EXISTS / LEFT JOIN anti-joins
 - Implement efficient keyset pagination
 - Understand when CTEs create optimization fences
-

Anti-Pattern Taxonomy

Performance Anti-Patterns (by impact):

CRITICAL (10x-1000x slowdown):

N+1 queries

Full table scans from missing indexes

Cartesian products (missing JOIN condition)

HIGH (5x-50x slowdown):

```
Functions on indexed columns (index bypass)
Correlated subqueries in SELECT list
OFFSET pagination on large tables
NOT IN with nulls
```

```
MODERATE (2x-10x slowdown):
  OR conditions preventing index usage
  Excessive DISTINCT
  SELECT *
  Large CTE optimization fences
```

```
LOW (< 2x slowdown, but important at scale):
  LIKE '%middle%' without trigram index
  COUNT(column) vs COUNT(*)
```

N+1 Query Pattern

The Problem

Application code fetches a list of records, then for each record executes a separate query — resulting in 1 + N queries.

```
# BAD: N+1 pattern in application code
orders = db.execute("SELECT id, customer_id FROM orders WHERE status = 'pending'")
for order in orders: # N orders
    customer = db.execute(
        "SELECT name FROM customers WHERE id = %s", order.customer_id
    ) # 1 query per order = N additional queries!
    print(f"Order {order.id} for {customer.name}")
# Total: 1 + N queries (N can be thousands)
```

The Fix: JOIN

```
-- GOOD: single query with JOIN
SELECT o.id, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'pending';
-- 1 query, uses indexes, optimal join algorithm
```

The Fix: IN clause for batch loading

```
-- When JOIN is not possible (e.g., across microservices):
-- BETTER: batch fetch with IN
SELECT id, name FROM customers WHERE id = ANY($1::int[]);
-- $1 = ARRAY of all customer_ids from the order list
-- One query instead of N
```

Detecting N+1

```
-- Find queries executed many times in succession with parameter variation
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
WHERE query LIKE 'SELECT%FROM customers WHERE id%'
AND calls > 1000
ORDER BY calls DESC;
```

SELECT * Anti-Pattern

The Problem

```
-- BAD: fetches all columns even if only 2 are needed
SELECT * FROM orders WHERE customer_id = 42;
-- Returns: id, customer_id, status, created_at, total, shipping_address,
billing_address,
--          notes, metadata, ... (100+ bytes per row)
```

The Problems with SELECT *

1. **Prevents Index Only Scans:** If only `id`, `status` are needed but `SELECT *`, must read heap
2. **More network bandwidth:** Sending unnecessary data
3. **More memory:** Larger rows in sort/hash operations
4. **Schema fragility:** Code breaks when columns are added/removed
5. **JSONB/TEXT columns:** Fetching large text/JSON columns unnecessarily

The Fix

```
-- GOOD: fetch only what you need
SELECT id, status, total FROM orders WHERE customer_id = 42;
-- Can use covering index: CREATE INDEX ON orders(customer_id) INCLUDE (id, status,
total)
-- → Index Only Scan, no heap access
```

Functions on Indexed Columns

The Problem

Applying a function to an indexed column prevents index usage:

```
-- BAD: lower() prevents index on email from being used
SELECT * FROM users WHERE lower(email) = 'alice@example.com';

-- BAD: date_trunc prevents index on created_at from being used
SELECT * FROM orders WHERE DATE(created_at) = '2024-06-15';
```

```
-- BAD: UPPER() prevents index usage
SELECT * FROM products WHERE UPPER(sku) = 'PRD-001';

-- BAD: type cast
SELECT * FROM orders WHERE customer_id::text = '42';

-- BAD: arithmetic
SELECT * FROM products WHERE price * 1.1 > 100;
```

The Fix: Rewrite Without Function (preferred)

```
-- GOOD: rewrite date function as range
WHERE created_at >= '2024-06-15' AND created_at < '2024-06-16'
-- Uses index on created_at

-- GOOD: store normalized values
WHERE email = lower('Alice@Example.COM') -- apply function to constant, not column
-- Uses index on email (if emails stored lowercase)

-- GOOD: rewrite arithmetic
WHERE price > 100 / 1.1 -- move the calculation to the constant side
-- Uses index on price
```

The Fix: Expression Index (if rewrite not possible)

```
-- Create expression index matching the function:
CREATE INDEX ON users(lower(email));
CREATE INDEX ON orders(DATE(created_at));
-- Now these queries use the expression index
```

OR vs UNION ALL for Index Usage

The Problem

OR conditions on different indexed columns often prevent efficient index usage:

```
-- BAD: OR on different columns – planner may choose Seq Scan
SELECT * FROM orders WHERE customer_id = 42 OR status = 'urgent';
-- No single index can satisfy both sides simultaneously
```

The Fix: UNION ALL

```
-- GOOD: split into two index-friendly queries
SELECT * FROM orders WHERE customer_id = 42
UNION ALL
SELECT * FROM orders WHERE status = 'urgent' AND customer_id != 42;
```

```
-- Each sub-query uses its own index
-- UNION ALL is faster than UNION (no deduplication)
```

When OR Works (Single Column)

```
-- This can use an index (planner may use BitmapOr):
SELECT * FROM orders WHERE status = 'pending' OR status = 'processing';
-- Better written as:
SELECT * FROM orders WHERE status IN ('pending', 'processing');
-- Both use the index on status via BitmapOr or Index Scan
```

NOT IN vs NOT EXISTS vs LEFT JOIN

This is a classic source of subtle bugs and performance issues.

NOT IN with NULLs (Bug!)

```
-- DANGER: NOT IN returns no rows if subquery contains any NULL!
SELECT * FROM orders
WHERE customer_id NOT IN (SELECT id FROM banned_customers);
-- If banned_customers contains any row with id IS NULL:
-- → returns ZERO rows (wrong!)
-- Because: x NOT IN (... , NULL) = UNKNOWN = false
```

NOT EXISTS (Preferred)

```
-- GOOD: NOT EXISTS correctly handles NULLs
SELECT * FROM orders o
WHERE NOT EXISTS (
    SELECT 1 FROM banned_customers bc WHERE bc.id = o.customer_id
);
-- If banned_customers has NULLs: correctly returns non-banned orders
-- Typically uses Anti-Join (most efficient)
```

LEFT JOIN / IS NULL (Also Good)

```
-- GOOD: anti-join via LEFT JOIN
SELECT o.*
FROM orders o
LEFT JOIN banned_customers bc ON o.customer_id = bc.id
WHERE bc.id IS NULL;
-- Returns orders whose customer is NOT in banned_customers
-- Uses Hash Anti-Join or Nested Loop Anti-Join
```

Performance Comparison

Method	NULL safe?	Performance	Notes
NOT IN	No	Can be slow (subquery evaluated once, but result set large)	Bug risk with NULLs
NOT EXISTS	Yes	Efficient (correlated but uses index)	Preferred
LEFT JOIN IS NULL	Yes	Efficient (hash anti-join)	Equivalent to NOT EXISTS

```
-- EXPLAIN comparison:
EXPLAIN SELECT * FROM orders WHERE customer_id NOT IN (SELECT id FROM vip_customers);
-- Shows: Seq Scan on orders, Nested Loop Anti-join or Seq Scan (potentially slow)

EXPLAIN SELECT * FROM orders o
WHERE NOT EXISTS (SELECT 1 FROM vip_customers v WHERE v.id = o.customer_id);
-- Shows: Hash Anti-join (efficient)
```

COUNT(*) vs COUNT(column) vs EXISTS

COUNT(*) vs COUNT(column)

```
-- COUNT(*): counts ALL rows (including NULLs in any column)
SELECT COUNT(*) FROM orders;

-- COUNT(column): counts NON-NULL values in column
SELECT COUNT(email) FROM users; -- excludes rows where email IS NULL
-- These are different and serve different purposes!

-- Performance: COUNT(*) uses the smallest available index
-- COUNT(column): must evaluate the column for each row (slightly slower)
```

EXISTS vs COUNT for "does anything exist?"

```
-- BAD: counts all matching rows just to check if any exist
IF (SELECT COUNT(*) FROM orders WHERE customer_id = 42) > 0:
    ...

-- GOOD: EXISTS stops at first match
IF EXISTS (SELECT 1 FROM orders WHERE customer_id = 42):
    ...

-- In SQL:
-- BAD:
SELECT CASE WHEN COUNT(*) > 0 THEN 'has orders' ELSE 'no orders' END
FROM orders WHERE customer_id = 42;
```

```
-- GOOD:
SELECT CASE WHEN EXISTS (SELECT 1 FROM orders WHERE customer_id = 42)
      THEN 'has orders' ELSE 'no orders' END;
```

Correlated Subqueries

The Problem

A correlated subquery in the SELECT list executes once per row:

```
-- BAD: correlated subquery executes for EVERY order
SELECT
  o.id,
  o.total,
  (SELECT COUNT(*) FROM order_items oi WHERE oi.order_id = o.id) AS item_count
FROM orders o
WHERE o.status = 'completed';
-- If 100K orders: 100K × subquery = extremely slow!
```

The Fix: Lateral Join or Aggregate Join

```
-- GOOD: single pass with LATERAL
SELECT o.id, o.total, oi_agg.item_count
FROM orders o
JOIN LATERAL (
  SELECT COUNT(*) AS item_count
  FROM order_items oi
  WHERE oi.order_id = o.id
) oi_agg ON true
WHERE o.status = 'completed';

-- OR: traditional aggregate join
SELECT o.id, o.total, COUNT(oi.id) AS item_count
FROM orders o
LEFT JOIN order_items oi ON oi.order_id = o.id
WHERE o.status = 'completed'
GROUP BY o.id, o.total;
```

When Correlated Subquery is Acceptable

```
-- For scalar lookups that are fast and highly selective:
SELECT o.id,
      (SELECT c.name FROM customers c WHERE c.id = o.customer_id) AS customer_name
FROM orders o
WHERE o.status = 'completed';
-- PostgreSQL may convert this to a hash join automatically (subquery collapse)
```

The planner often converts correlated subqueries to joins automatically. Check EXPLAIN to verify.

Excessive DISTINCT

The Problem

`DISTINCT` sorts and deduplicates the entire result — $O(N \log N)$ work.

```
-- BAD: fetching all rows then deduplicating
SELECT DISTINCT customer_id FROM orders;
-- Reads all orders, sorts, deduplicates
-- 1M orders with 100K customers → lots of wasted work
```

When DISTINCT Indicates a Bad Query

If you need `DISTINCT` because of a `JOIN` that produces duplicates:

```
-- BAD: JOIN produces duplicates, DISTINCT removes them
SELECT DISTINCT c.id, c.name
FROM customers c
JOIN orders o ON c.id = o.customer_id;
-- If customers have many orders, this produces duplicates then removes them!

-- GOOD: use EXISTS to avoid duplicates from the start
SELECT c.id, c.name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
-- No duplicates to remove → no need for DISTINCT
```

Using LIMIT without ORDER BY

The Problem

`LIMIT` without `ORDER BY` returns arbitrary rows. This is a logical bug (inconsistent results) AND a potential performance issue.

```
-- BUG: returns different rows each run
SELECT * FROM orders WHERE status = 'pending' LIMIT 10;
-- Each run may return different rows!
-- Also: planner doesn't know which rows you want, can't optimize

-- CORRECT: always include ORDER BY with LIMIT
SELECT * FROM orders WHERE status = 'pending'
ORDER BY created_at DESC -- deterministic order
LIMIT 10;
-- Planner can use index to return top 10 without sorting everything
```

STRING_AGG / ARRAY_AGG Performance

Large Aggregation Result

```
-- BAD: concatenating huge strings
SELECT customer_id, STRING_AGG(product_name, ', ' ORDER BY product_name)
FROM orders
GROUP BY customer_id;
-- If one customer has 10,000 orders: STRING_AGG builds a 100KB string!

-- BETTER: paginate or limit
SELECT customer_id,
       STRING_AGG(product_name, ', ' ORDER BY product_name)
FROM (
  SELECT customer_id, product_name
  FROM orders
  WHERE customer_id = 42 -- filter first
  LIMIT 100             -- cap the result set
) limited
GROUP BY customer_id;
```

Array Aggregation for Batch Lookups

```
-- GOOD: use ARRAY_AGG to batch a lookup (avoids N+1)
SELECT c.id, ARRAY_AGG(o.id ORDER BY o.created_at DESC) AS recent_order_ids
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE c.type = 'premium'
GROUP BY c.id;
```

LIKE '%prefix%' and Substring Search

The Problem

LIKE '%middle%' cannot use a standard B-tree index:

```
-- BAD: full table scan
SELECT * FROM products WHERE name LIKE '%laptop%';
SELECT * FROM products WHERE name ILIKE '%dell%';
```

The Fix: pg_trgm Extension

```
-- Install trigram extension
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Create GIN trigram index
CREATE INDEX ON products USING GIN (name gin_trgm_ops);

-- Now LIKE '%middle%' and ILIKE use the index:
```

```
SELECT * FROM products WHERE name LIKE '%laptop%'; -- uses index
SELECT * FROM products WHERE name ILIKE '%DELL%'; -- uses index
SELECT * FROM products WHERE name ~ 'dell.*laptop'; -- regex, uses index!
```

For Prefix-only LIKE

```
-- Standard B-tree handles prefix LIKE:
CREATE INDEX ON products(name);
SELECT * FROM products WHERE name LIKE 'Dell%'; -- uses B-tree index
```

Pagination Anti-Pattern (OFFSET)

The Problem

```
-- BAD: OFFSET pagination — gets worse as pages increase
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 0; -- page 1 (fast)
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 100; -- page 6 (OK)
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 10000; -- page 501
(slow!)
-- Must read and discard 10,000 rows before returning 20
-- PostgreSQL cannot skip to page N without reading 1..N
```

The Fix: Keyset Pagination

```
-- GOOD: keyset (cursor) pagination
-- Page 1:
SELECT id, title, created_at FROM posts
ORDER BY created_at DESC, id DESC
LIMIT 20;
-- Returns last row: created_at='2024-06-15', id=12345

-- Page 2 (using last row's values as cursor):
SELECT id, title, created_at FROM posts
WHERE (created_at, id) < ('2024-06-15', 12345) -- continue from cursor
ORDER BY created_at DESC, id DESC
LIMIT 20;
-- Uses composite index on (created_at DESC, id DESC)
-- Jumps directly to the cursor position: O(log N) regardless of page number!
```

Keyset Pagination Requirements

1. Order must be deterministic (include unique column like `id` as tiebreaker)
2. Need an index matching the ORDER BY
3. Cannot jump to arbitrary page numbers (only forward/backward)

CTE Optimization Fence

The Problem

In PostgreSQL < 12, CTEs (WITH clauses) were always **optimization fences** — the planner could not push predicates through them, preventing index usage.

```
-- PRE-PG12 behavior (still relevant for older syntax):
WITH recent_orders AS (
    SELECT * FROM orders WHERE created_at > '2024-01-01'
)
SELECT * FROM recent_orders WHERE customer_id = 42;
-- Pre-PG12: scans ALL recent orders, then filters by customer_id
-- PostgreSQL 12+: planner can push customer_id filter into the CTE
```

PostgreSQL 12+ (Inlined CTEs)

```
-- PG12+: CTEs are inlined by default (NOT optimization fences)
WITH recent_orders AS (
    SELECT * FROM orders WHERE created_at > '2024-01-01'
)
SELECT * FROM recent_orders WHERE customer_id = 42;
-- PG12+: equivalent to:
-- SELECT * FROM orders WHERE created_at > '2024-01-01' AND customer_id = 42
-- Can use composite index (customer_id, created_at) efficiently!
```

Forcing a Fence (when you want it)

```
-- Force materialization (useful for recursive CTEs or when CTE result reused
multiple times):
WITH expensive_computation AS MATERIALIZED (
    SELECT * FROM large_table WHERE complex_condition
)
SELECT * FROM expensive_computation WHERE type = 'A'
UNION ALL
SELECT * FROM expensive_computation WHERE type = 'B';
-- MATERIALIZED: run once, store result, reuse — avoids computing twice
```

Common Mistakes

1. Not using EXISTS for "does row exist" checks

```
-- Slow:
IF (SELECT COUNT(*) FROM cache WHERE key = 'x') > 0 THEN ...
-- Fast:
IF EXISTS (SELECT 1 FROM cache WHERE key = 'x') THEN ...
```

2. Using OR for multi-value equality (use IN instead)

```
-- OK but verbose:
WHERE status = 'a' OR status = 'b' OR status = 'c'
-- Better:
WHERE status IN ('a', 'b', 'c')
-- Both can use the same index, but IN is cleaner
```

3. Not using row value comparison for keyset pagination

```
-- OK but verbose:
WHERE (created_at < cursor_date OR (created_at = cursor_date AND id < cursor_id))
-- Cleaner and equivalent:
WHERE (created_at, id) < (cursor_date, cursor_id)
```

4. Using DISTINCT when JOIN + GROUP BY would be cleaner

```
-- Ugly and potentially slow:
SELECT DISTINCT c.id FROM customers c JOIN orders o ON c.id = o.customer_id;
-- Better:
SELECT c.id FROM customers c WHERE EXISTS (SELECT 1 FROM orders WHERE customer_id = c.id);
```

Best Practices

1. **Always JOIN instead of N+1** — fix at application level, not database level.
2. **Select only needed columns** — enables covering indexes and reduces I/O.
3. **Move functions from columns to constants** — prevents index bypass.
4. **Use keyset pagination** for user-facing paginated lists with many pages.
5. **Use NOT EXISTS instead of NOT IN** — safer (handles NULLs) and often faster.
6. **Use EXPLAIN (ANALYZE, BUFFERS)** after rewriting to verify improvement.
7. **Check CTEs for optimization fence behavior** — use `MATERIALIZED` intentionally.

Interview Questions & Answers

Q1. What is N+1 query problem and how do you fix it?

A: N+1 is when application code executes 1 query to fetch a list of N rows, then for each row executes 1 more query — totaling N+1 queries. For N=1000, this means 1001 round trips to the database instead of 1. The fix is to JOIN the related data in the initial query (1 query), or batch-fetch related data with `IN (all_ids)` (2 queries). The problem is common with ORM frameworks that lazy-load associations.

Q2. Why does NOT IN fail when the subquery contains NULLs?

A: SQL's three-valued logic: `x NOT IN (... , NULL)` evaluates as `x != NULL` which is `UNKNOWN` (not `FALSE`), so the entire `NOT IN` condition is `UNKNOWN` — which is treated as `FALSE` in `WHERE` clauses. If a subquery returns any `NULL`, `NOT IN` returns no rows. `NOT EXISTS` correctly handles `NULL`s: the subquery either finds a matching non-`NULL` row (`EXISTS = true`, `NOT EXISTS = false`) or finds no matching row (`NOT EXISTS = true`). Always prefer `NOT EXISTS` over `NOT IN` for anti-joins.

Q3. What is a CTE optimization fence and when does it matter?

A: A CTE optimization fence is when the query planner cannot push predicates from the outer query into a CTE. The CTE is materialized first (all rows computed and stored), then the outer filter is applied. This prevents index usage for predicates on the CTE result. Pre-PostgreSQL 12, all CTEs were fences. PG12+ inlines CTEs by default (no fence), allowing the planner to push predicates through. Use `WITH cte AS MATERIALIZED (...)` to explicitly force materialization when the CTE result is expensive and reused multiple times.

Exercises with Solutions

Exercise 1

Rewrite this N+1 pattern:

```
users = query("SELECT id, name FROM users WHERE active = true")
for user in users:
    count = query("SELECT COUNT(*) FROM orders WHERE user_id = %s", user.id)
    print(f"{user.name}: {count} orders")
```

Solution:

```
-- Single query replacing the N+1 pattern:
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE u.active = true
GROUP BY u.id, u.name
ORDER BY u.name;
```

Exercise 2

Fix this pagination query for page 500 (OFFSET 9980):

```
SELECT * FROM blog_posts ORDER BY published_at DESC LIMIT 20 OFFSET 9980;
```

Solution:

```
-- Add index:
CREATE INDEX ON blog_posts(published_at DESC, id DESC);

-- Keyset pagination (requires storing cursor from previous page):
-- After page 499, last row had: published_at='2023-12-01', id=45678

-- Page 500:
```

```
SELECT * FROM blog_posts
WHERE (published_at, id) < ('2023-12-01', 45678)
ORDER BY published_at DESC, id DESC
LIMIT 20;
-- O(log N) instead of O(N) – works identically fast for page 1 and page 500
```

Production Scenarios

Scenario 1: API Endpoint 5-Second Response Time

A REST API endpoint `/api/orders?customer_id=42` was taking 5 seconds. Investigation found the code was doing:

```
order_ids = db.execute("SELECT id FROM orders WHERE customer_id = 42")
for oid in order_ids: # 200 orders
    items = db.execute("SELECT * FROM order_items WHERE order_id = %s", oid)
    ...
```

Fix:

```
-- Replace 201 queries with 1:
SELECT o.id, oi.product_id, oi.quantity, oi.unit_price
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
WHERE o.customer_id = 42
ORDER BY o.id;
-- API response time: 5 seconds → 15ms
```

Cross-References

- [01 explain basics.md](#) — Verifying rewrites with EXPLAIN
- [03 scan types.md](#) — Understanding scan choice after rewrites
- [09 slow query analysis.md](#) — Finding N+1 and other anti-patterns
- [10 optimization cookbook.md](#) — More rewrite scenarios
- [../07 Indexes/11 expression indexes.md](#) — Fixing function-on-column patterns